NED Academy

PYTHON LECTURE 2

Strings & Conditional Statements

Comprehensive Study Notes + Worked Example Programs

Study guide
Beginner-friendly notes on strings and conditional statements with original worked examples, explained concepts, and practice programs.

Designed for beginner Python learners

Learning Outcomes

- Define Python strings and create them using single, double, and triple quotes.
- Use escape sequences, concatenation, len(), indexing, slicing, and negative indexing.
- Explain why strings are immutable and use common string methods safely.
- Apply if, elif, and else statements with correct Python indentation.
- Use comparison, logical, and modulo operators in decision-making programs.
- Build programs for grading, voting/driving checks, odd/even detection, greatest-number selection, and divisibility tests.

Contents

Part	Coverage
A. Strings	Creation, quoting, escape sequences, concatenation, length, indexing, immutability, slicing, negative indexing, methods
B. Conditional Statements	if, elif, else, indentation, Boolean conditions, logical operators, nesting
C. Worked Programs	Name length, dollar count, traffic light, voting, grading, odd/even, largest numbers, multiple of 7
D. Extra Practice	Additional beginner programs and review questions

Important
Run every example yourself. Change the input values and predict the output before executing the program.

Part A - Python Strings

Core idea
A string is a sequence of characters used to store text. A string may contain one character, a word, a sentence, spaces, digits, punctuation, or multiple lines.

1. Creating Strings

- Double quotes: useful for ordinary text and especially convenient when the text contains an apostrophe.
- Single quotes: equally valid for many strings.
- Triple quotes: useful for multi-line text or long blocks of text.

Example: three ways to create strings

```
string_one = "Python"
string_two = 'College'
string_three = """This is line 1.
This is line 2.
This is line 3."""

print(string_one)
print(string_two)
print(string_three)
```

2. Why Different Quote Types Matter

A common beginner problem occurs when the quotation mark used to delimit the string also appears inside the text. Choose the outer quotes so the inner apostrophe or quote does not terminate the string early.

```
message = "It's a Python tutorial."
print(message)

# Another valid approach using an escape character:
message2 = 'It\'s a Python tutorial.'
print(message2)
```

Beginner tip
If a sentence contains an apostrophe such as It's, using double quotes around the complete string is often the simplest option.

3. Escape Sequence Characters

Escape sequence	Purpose
\n	Move output to a new line
\t	Insert a horizontal tab space
\\	Insert a backslash character

Escape sequence	Purpose
\'	Insert a single quote inside a single-quoted string
\"	Insert a double quote inside a double-quoted string

Example: formatted string output

```
print("Python\nProgramming")
print("Name:\tAli")
print("Path: C:\\Users\\Student")
```

Expected idea
The first print displays text on two lines; the second inserts a tab between the label and value.

4. String Concatenation

Concatenation means joining strings together. Python uses the + operator for string concatenation.

```
first = "Data"
second = "Science"
final_string = first + " " + second
print(final_string)    # Data Science
```

5. Finding String Length with len()

- len(string) returns the number of characters in a string.
- Spaces count as characters.
- Digits and symbols also count as characters.

```
text = "Data Science"
print(len(text))

symbol_text = "Rs. 500!"
print(len(symbol_text))
```

Remember
The len() result is a count. String indexes still start from 0, so the last positive index is len(string) - 1.

6. String Indexing

Each character has a position called an index. Python uses zero-based indexing: the first character is index 0, the second is index 1, and so on.

```
text = "Python"
print(text[0])   # P
print(text[1])   # y
print(text[5])   # n
```

Character	P y t h o n
Positive index	0 1 2 3 4 5
Negative index	-6 -5 -4 -3 -2 -1

7. Strings Are Immutable

Immutable means that an existing string cannot be changed character-by-character. You may create a new string, but you cannot assign a new character directly to a position in the old string.

```
name = "Python"
# name[0] = "J"     # TypeError: strings do not support item assignment

# Correct approach: create a new value
name = "J" + name[1:]
print(name)      # Jython
```

8. String Slicing

Syntax

string[start:end] - start is included; end is excluded.

```
word = "college"
print(word[1:4])   # oll
print(word[0:3])   # col
print(word[3:])    # lege
print(word[:4])    # coll
print(word[:])     # college
```

- If start is omitted, Python assumes 0.
- If end is omitted, Python continues to the end of the string.
- Slicing is very useful when selecting parts of text or structured data.

9. Negative Indexing and Slicing

Negative indexes count from the end. Index -1 refers to the last character, -2 to the second-last character, and so on.

```
word = "Python"
print(word[-1])     # n
print(word[-3:-1])  # ho
print(word[-4:])    # thon
```

Rule still applies

Even with negative slicing, the end position is excluded.

10. Useful String Methods

Method	What it does
endswith(value)	Returns True if the string ends with the given value; otherwise False.
capitalize()	Returns a copy with the first character capitalized and remaining characters adjusted accordingly.
replace(old, new)	Returns a new string with occurrences of old replaced by new.
find(value)	Returns the index of the first occurrence; returns -1 if not found.
count(value)	Returns how many times the value appears.

One program demonstrating the main methods

```
text = "i am studying Python"

print(text.endswith("Python"))
print(text.capitalize())
print(text.replace("Python", "JavaScript"))
print(text.find("studying"))
print(text.find("q"))
print(text.count("i"))
```

Important concept
Methods such as capitalize() and replace() return new strings. The original string remains unchanged unless you assign the result back to the variable.

Reassigning a changed string

```
text = "python is useful"
text = text.capitalize()
print(text)
```

11. Practice Programs - Strings

Program 1 - Input a first name and print its length

```
first_name = input("Enter your first name: ")
print("Your name has", len(first_name), "characters.")
```

Program 2 - Count dollar signs in a string

```
text = input("Enter a sentence containing $ signs: ")
count_dollar = text.count("$")
print("Number of $ signs:", count_dollar)
```

Program 3 - Full string analysis

```
text = input("Enter some text: ")

print("Length:", len(text))
print("First character:", text[0] if text else "No character")
print("Last character:", text[-1] if text else "No character")
print("Uppercase copy:", text.upper())
print("Number of spaces:", text.count(" "))
```

Why this extra program helps

It combines input, len(), indexing, a string method, and a simple safeguard for empty input.

Part B - Conditional Statements

Core idea

Conditional statements allow a program to choose what to do based on whether a condition is True or False.

12. The if Statement

```
age = 21

if age >= 18:
    print("Eligible to apply for a driving license")
    print("Eligible to vote")
```

- The condition follows if.
- A colon : ends the condition line.
- Indented lines belong to the if block.
- The block runs only when the condition evaluates to True.

13. Indentation in Python

Python uses indentation to define code blocks. Unlike languages that use curly braces, Python relies on consistent spaces. Four spaces is the standard convention.

```
temperature = 30
if temperature > 25:
    print("It is warm")
    print("Drink water")
print("Program finished")
```

Notice

Only the two indented lines depend on the condition. The final line runs regardless of the condition.

14. if - elif - else

```
light = "yellow"

if light == "red":
    print("Stop")
elif light == "yellow":
    print("Get ready")
elif light == "green":
    print("Go")
else:
    print("Light is broken or invalid")
```

- if checks the first condition.
- elif checks another condition only if the earlier branch was False.
- else runs when none of the earlier conditions are True.
- In an if/elif/else chain, the first True branch runs and the rest are skipped.

15. Independent if vs elif Chain

Independent if statements	if / elif chain
Every if condition can be checked.	Later conditions are checked only if earlier ones fail.
More than one block may run.	Normally only one branch runs.
Use when conditions are independent.	Use when choices are mutually exclusive.

```
score = 95

# Independent checks
if score >= 50:
    print("Pass")
if score >= 90:
    print("Excellent")

# Mutually exclusive chain
if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
else:
    print("Below B")
```

16. Operators Commonly Used in Conditions

Operator	Meaning
==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
and	both conditions must be True
or	at least one condition must be True

Operator	Meaning
not	reverses a Boolean result
%	modulo - gives the remainder

17. Grade Assignment Program

The lecture demonstrates assigning grades by checking score ranges. A clean Python approach is to order the conditions from highest to lowest; then you do not need to repeat the upper bound in every branch.

```
marks = float(input("Enter marks: "))

if marks >= 90:
    grade = "A"
elif marks >= 80:
    grade = "B"
elif marks >= 70:
    grade = "C"
else:
    grade = "D"

print("Grade:", grade)
```

Equivalent range logic

A branch such as marks >= 80 and marks < 90 is valid. Ordering the elif chain from highest to lowest makes the same logic shorter and easier to read.

18. Input - Operation - Output

- Input: receive a value from the user or another source.
- Operation: apply logic, calculations, comparisons, or transformations.
- Output: display or return the result.

```
# Input
marks = 94

# Operation
if marks >= 90:
    result = "A"
else:
    result = "Below A"

# Output
print(result)
```

19. Nested Conditional Statements

Nesting means placing one conditional statement inside another. It is useful when the second decision only makes sense after the first condition has already been satisfied.

```
age = int(input("Enter age: "))

if age >= 18:
    if age <= 80:
        print("Age is within the example driving range")
    else:
        print("Age is above the example upper limit")
else:
    print("Too young for the example driving condition")
```

Design tip
Use nesting when there is a true parent-child relationship between decisions. If the conditions are simply alternative choices, an elif chain is often clearer.

Part C - Worked Example Programs

20. Voting Eligibility

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")
```

21. Odd or Even Number

An even number leaves remainder 0 when divided by 2. The modulo operator % is used to check the remainder.

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")
```

22. Greatest Among Three Numbers

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b and a >= c:
    print("Largest number is:", a)
elif b >= c:
    print("Largest number is:", b)
```

```
else:
    print("Largest number is:", c)
```

Why >= is useful

Using >= handles ties more safely than only using >. If two numbers are equal and are the largest, the program still identifies a largest value.

23. Greatest Among Four Numbers - Homework Extension

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
d = float(input("Enter fourth number: "))

if a >= b and a >= c and a >= d:
    largest = a
elif b >= c and b >= d:
    largest = b
elif c >= d:
    largest = c
else:
    largest = d

print("Largest number is:", largest)
```

Alternative

Python also has max(a, b, c, d), but the conditional version is better practice for learning if/elif/else logic.

24. Check Whether a Number Is a Multiple of 7

```
x = int(input("Enter a number: "))

if x % 7 == 0:
    print("Multiple of 7")
else:
    print("Not a multiple of 7")
```

25. General Divisibility Checker

```
number = int(input("Enter a number: "))
divisor = int(input("Enter a divisor: "))

if divisor == 0:
    print("Cannot test divisibility by zero.")
elif number % divisor == 0:
    print(number, "is a multiple of", divisor)
else:
    print(number, "is not a multiple of", divisor)
```

26. Traffic Light Program

```
light = input("Enter light color: ").strip().lower()

if light == "red":
    print("Stop")
elif light == "yellow":
    print("Get ready")
elif light == "green":
    print("Go")
else:
    print("Invalid or broken traffic light")
```

27. Simple Admission Decision

```
marks = float(input("Enter marks: "))
attendance = float(input("Enter attendance percentage: "))

if marks >= 60 and attendance >= 75:
    print("Eligible")
else:
    print("Not eligible")
```

Extra example

This shows the logical operator and: both conditions must be True for the eligibility message.

28. Positive, Negative, or Zero

```
number = float(input("Enter a number: "))

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Part D - Combined Practice and Revision

29. Password Length Checker

```
password = input("Create a password: ")

if len(password) >= 8:
    print("Password length is acceptable")
else:
```

```
    print("Password must contain at least 8 characters")
```

30. Name Initials Using Indexing

```
first_name = input("First name: ").strip()
last_name = input("Last name: ").strip()

if first_name and last_name:
    initials = first_name[0].upper() + last_name[0].upper()
    print("Initials:", initials)
else:
    print("Please enter both names")
```

31. Check String Ending

```
filename = input("Enter a filename: ").strip()

if filename.lower().endswith(".py"):
    print("This looks like a Python source file")
else:
    print("This is not a .py filename")
```

32. Find and Replace in User Text

```
text = input("Enter text: ")
old_word = input("Word to replace: ")
new_word = input("Replacement word: ")

if text.find(old_word) != -1:
    updated = text.replace(old_word, new_word)
    print("Updated text:", updated)
else:
    print("The word was not found")
```

33. Common Beginner Mistakes

Mistake	Correction
Using = instead of == in a condition	Use == to compare values; = assigns a value.
Forgetting the colon after if/elif/else	End the condition line with :
Incorrect indentation	Indent the body consistently, usually four spaces.
Trying to change string[index] directly	Strings are immutable; build a new string instead.
Forgetting that slice end is excluded	Remember string[start:end] stops before end.
Assuming find() returns False when absent	find() returns -1 when the value is not found.
Not converting numeric input	Use int(input(...)) or float(input(...)) when doing numeric comparisons.

34. Quick Revision Sheet

Concept	Memory aid
String	A sequence of characters inside quotes.
Concatenation	Join strings with +.
len()	Count characters, including spaces and symbols.
Indexing	Single character; starts at 0.
Slicing	Part of a string; start included, end excluded.
Negative index	Count backward from the end; -1 is last.
Immutable	Existing characters cannot be changed by index assignment.
if	Run a block when a condition is True.
elif	Try another condition after earlier conditions fail.
else	Fallback when no previous condition is True.
and	Both conditions must be True.
%	Remainder; useful for odd/even and divisibility.

35. Self-Check Questions

1. What is the difference between indexing and slicing?
2. Why is the last positive index of a string len(text) - 1?
3. What does text.find("x") return if x is absent?
4. Why can text[0] = "A" fail?
5. What is the difference between two independent if statements and an if/elif chain?
6. Why does number % 2 == 0 indicate an even number?
7. What happens if Python indentation is inconsistent?
8. Write a program that checks whether a number is a multiple of both 3 and 5.
9. Write a program that checks whether a username has at least 5 characters.
10. Write a program that prints the middle three characters of a 5-character string.

36. Suggested Lab Tasks

1. Create three strings using single, double, and triple quotes and print them.
2. Take a full name as input and print its length, first character, and last character.
3. Create a string and demonstrate at least four different slices.
4. Demonstrate endswith(), capitalize(), replace(), find(), and count() on your own sentence.
5. Build a grade calculator using if/elif/else.
6. Build an odd/even checker using modulo.
7. Find the largest among four user-entered numbers without using max().
8. Create a divisibility checker for any user-supplied divisor.

Study strategy
First write the logic in plain English, then convert it into Python syntax. Test at least three different inputs for each program.

37. Final Practice Advice

- Run each program yourself and test multiple inputs.
- Try changing comparison operators to see how output changes.
- Create your own examples using strings, slicing, and conditions.
- Trace each program line by line before executing it.

End of notes

Practice the programs by changing inputs, predicting outputs, and deliberately introducing small errors to understand how Python responds.